

SOFTWARE & SYSTEM DESIGN · EPISODE 01

Decoupling LLM Providers from Business Logic

Engineering production-ready AI systems from first principles.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

The Problem

Many teams build initial features by calling raw AI SDKs directly inside their web routers or service classes. What begins as a fast integration quickly degrades as the application scales.

Tightly coupling third-party APIs directly to your core business logic renders code brittle, impossible to test reliably in isolation, and restricts you to a single vendor.

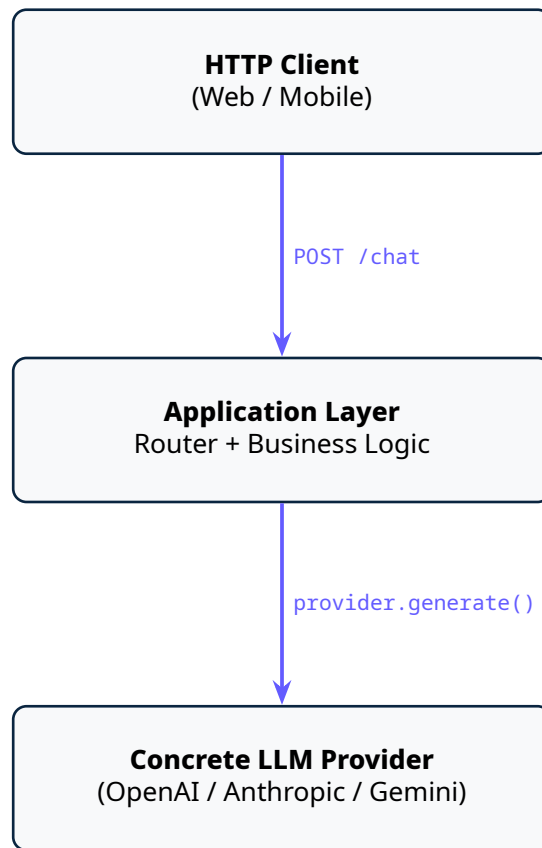
The Naive Request Lifecycle

A Tightly Coupled Architecture

What is Tight Coupling?

Coupling measures how much one software component depends on another.

A system is **tightly coupled** when it depends on **concrete implementations** rather than **abstractions**.



Why is this an Anti-pattern?

- Business logic knows the SDK.
- Provider changes ripple through application code.
- Difficult to unit test because vendor SDKs must be mocked.
- Creates vendor lock-in and reduces maintainability.

From Architecture to Code

The previous diagram showed a tightly coupled architecture.

Now let's see what that dependency looks like in actual production code. Every highlighted line below increases coupling between the application and the external LLM provider.

app/api/routes/copilot.py

```
from fastapi import APIRouter

!!! Vendor SDK imported directly into the application layer
from openai import OpenAI // Or any LLM Provider

router = APIRouter()

!!! Global provider instance bound to OpenAI
client = OpenAI(api_key="sk-...")

@router.post("/chat")
async def chat(prompt: str):

    !!! Business logic knows the OpenAI API
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "user", "content": prompt}
        ]
    )

    !!! OpenAI response model leaks into business logic
    return {
        "response": response.choices[0].message.content
    }
```

Why is this a Bad Architecture?

- **The router imports the vendor SDK directly.**

The application layer depends on a specific LLM provider rather than an abstraction, creating tight coupling.

- **Business logic understands provider-specific APIs.**

Methods calls such as `chat.completions.create()` become part of core logic, binding functionality to one provider's interface.

- **Provider response objects leak upward.**

Application logic directly consumes provider-specific response formats (`response.choices[0].message.content`), Any change in the provider's schema propagates through routers, services, and tests, forcing widespread code modifications.

- **Testing becomes unnecessarily difficult.**

Unit tests must mock full SDK clients and replicate provider-specific response structures, adding unnecessary overhead.

- **Swapping providers requires code changes.**

Migrating to Anthropic, Gemini, Azure OpenAI, or a self-hosted model demands application changes instead of a simple implementation replacement.

The Core Problem

Every layer of the application depends on a **concrete LLM provider**. To eliminate this dependency, we need an **abstraction** that sits between our business logic and every external AI SDK.

Let's build that abstraction.

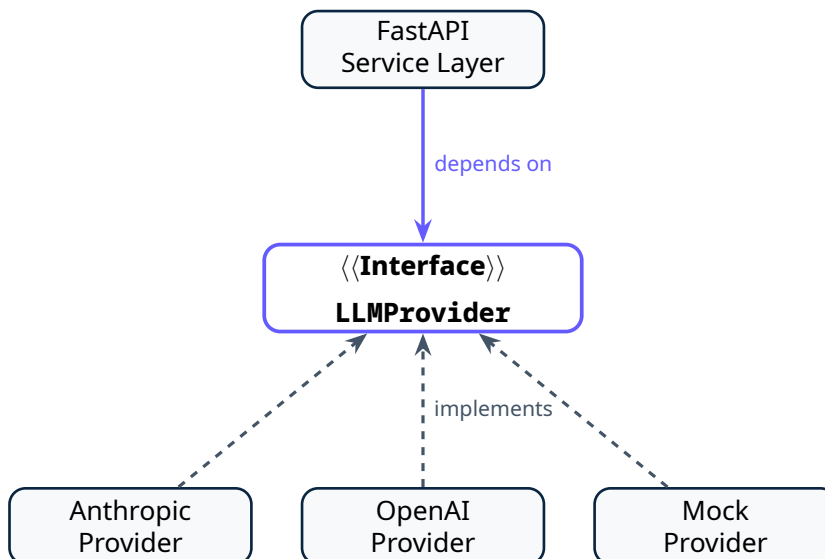
The Boundary Interface

The solution is to introduce a stable boundary between our application and external AI providers.

Instead of depending directly on OpenAI, Anthropic, or Gemini, the application depends on a single interface:

LLMProvider

Every provider simply implements this contract.



By depending only on an abstraction, the application remains unchanged when switching providers, upgrading SDKs, or introducing mock implementations for testing.

Defining the Contract

So far, our business logic has depended directly on a concrete LLM provider. That tightly couples the application to a single vendor, making provider changes ripple through the codebase.

To solve this, we introduce a simple contract that every LLM provider must implement. Whether the implementation wraps OpenAI, Anthropic, Gemini, or a local model, the application always interacts with the same interface, providing a uniform language across all providers.

The contract has a single responsibility: given a list of messages, return the generated response. By depending on this abstraction instead of a concrete SDK, our business logic remains provider-independent while ensuring consistent behavior regardless of the underlying implementation.

app/providers/interfaces.py

```
from abc import ABC, abstractmethod

class LLMProvider(ABC):
    @abstractmethod
    async def generate(self, messages: list[dict]) -> str:
        """The standard input/output boundary."""
        pass
```

Our application now depends on an abstraction, not an SDK.

Every provider must implement the same interface before it can be used. The application never knows which provider is underneath, it only knows the contract.

Implementing the Contract

With the contract in place, every provider only needs to implement the same interface. Each implementation translates our standard contract into the provider's SDK, keeping vendor-specific code isolated behind a single class.

This separation makes each provider easy to unit test while future-proofing the architecture. If a provider changes its SDK, only that implementation needs to be updated. The rest of the application remains untouched.

app/providers/openai_provider.py

```
from openai import AsyncOpenAI
from app.providers.interfaces import LLMProvider

class OpenAIProvider(LLMProvider):
    def __init__(self):
        self.client = AsyncOpenAI()

    async def generate(self, messages: list[dict]) -> str:
        # Vendor-specific logic stays isolated here
        response = await self.client.chat.completions.create(
            model="gpt-4o",
            messages=messages
        )
        return response.choices[0].message.content
```

The application depends on the interface, while each provider handles its own SDK internally.

The final step is injecting the provider implementation as a dependency into our application.

The Service Depends on an Abstraction

We now have an `OpenAIProvider` that implements the `LLMProvider` contract. The remaining question is: how does our application receive this implementation?

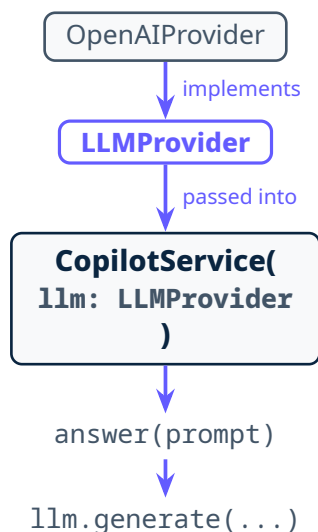
Instead of creating an `OpenAIProvider` directly, the `CopilotService` simply requests an `LLMProvider`. Dependency Injection supplies the concrete implementation at runtime, so the service never knows which provider is underneath. It only knows how to call `generate(messages)` through the contract.

`app/services/copilot_service.py`

```
from app.providers.interfaces import LLMProvider

class CopilotService:
    def __init__(self, llm: LLMProvider):
        self.llm = llm

    async def answer(self, prompt: str):
        return await self.llm.generate([
            {"role": "user", "content": prompt}
        ])
```



Constructor Injection

The constructor receives an `LLMProvider` as an argument. This injected dependency can be any class that implements the interface, keeping the service independent of a specific LLM provider.

The Framework Wires Everything Together

Our service expects an `LLMProvider`, but it never creates one. Instead, it declares the dependency and lets the framework provide it.

This is where **FastAPI** acts as the composition root (the single place where concrete implementations are created and wired together). Using `Depends()`, FastAPI resolves the dependency, calls the provider function, and injects the resulting `LLMProvider` into the service.

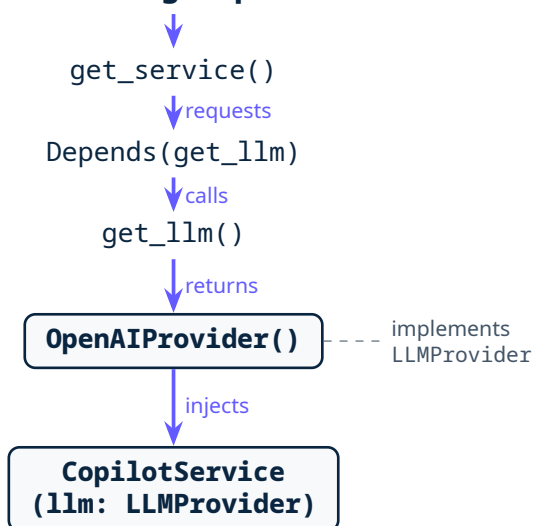
app/dependencies.py

```
from fastapi import Depends
from app.providers.openai_provider import OpenAIProvider

def get_llm():
    return OpenAIProvider()

def get_service(
    llm = Depends(get_llm)
):
    return CopilotService(llm)
```

Incoming Request



FastAPI Dependency Injection

`Depends()` tells FastAPI how to resolve a dependency. When an `LLMProvider` is required, FastAPI calls the provider function, obtains the concrete implementation (such as `OpenAIProvider`), and injects it automatically into the service.

Dependency Injection at Runtime

The framework automatically creates the concrete implementation and injects it into the service just in time to handle the request.

Production Capabilities Enabled by the Abstraction

Because every request flows through the same `LLMProvider` interface, the provider boundary becomes the single extension point for the entire application. Instead of modifying business logic, we extend the provider layer with new capabilities while keeping the service layer unchanged.

- **Single abstraction point**

Minimize the surface area of change. Every request flows through `LLMProvider`. One boundary absorbs volatility.

- **Centralized production behavior**

Separate cross-cutting concerns from business logic. Retries, failover, logging, caching, cost tracking, and rate limiting all live at the provider boundary. Keep business code pure.

- **Decorator pattern**

Composition over inheritance. Wrappers like `LoggingProvider` or `RetryProvider` compose around the core provider. Behaviors are layered without modifying the core.

- **Replaceability**

Depend on abstractions, not implementations. You can swap OpenAI for Anthropic, or add/remove wrappers, without touching `CopilotService`. The service remains blind to vendor quirks.

- **Future-proofing**

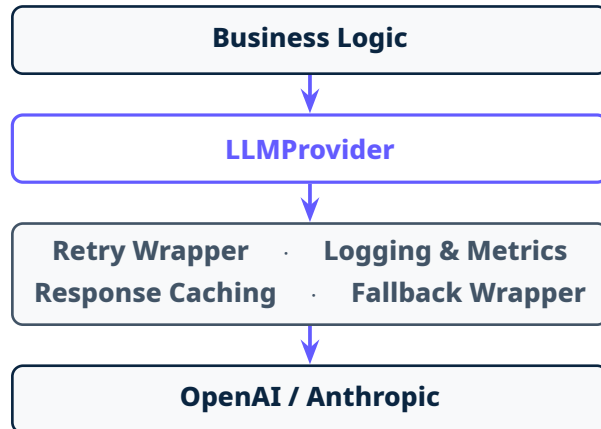
Extend without modifying. New capabilities (e.g., caching, observability) can be added by introducing another wrapper. The abstraction boundary enables open/closed design.

Extending the Boundary in Practice

By utilizing the decorator pattern, we can stack these production capabilities around our core provider.

Each wrapper implements the same `LLMProvider` interface, allowing them to be nested infinitely.

The business logic calls the interface, the request passes transparently down the chain, and the response bubbles back up.



Adding Capabilities via the Decorator Pattern

```
class LoggingProvider(LLMProvider):
    def __init__(self, inner: LLMProvider):
        self.inner = inner # Wraps the next provider in the chain

    async def generate(self, messages: list[dict]) -> str:
        logger.info(f"LLM Request: {messages}")

        # Passes the request down the chain
        response = await self.inner.generate(messages)

        logger.info(f"LLM Response: {response}")
        return response
```

Lessons Learned

Flexibility

Switch providers effortlessly without changing business logic.

Maintainability

New providers only need to implement one interface.

Scalability

Production features remain centralized and organized.

Reliability

Retries, fallback, and monitoring become reusable.

THE ARCHITECTURAL PRINCIPLE

**Good architecture isn't
about choosing the right
SDK.**

**It's about making the SDK
replaceable.**

Wrap it. Abstract it. Inject it.

COMING UP NEXT...

Clean Persistence: The Repository Pattern in FastAPI

Learn how the Repository Pattern separates persistence from business logic, enabling fast, database-independent unit tests and a cleaner architecture.

Building production-grade AI systems, one architectural pattern at a time.

github.com/MrHeaven1y
linkedin.com/in/dibayendu-mukherjee-bb897b267

Dibayendu Mukherjee